Listing 6.1 `first_btree.py`

```
import py_trees

class HasApple(py_trees.behaviour.Behaviour):      #1
    def __init__(self, name):
        super(HasApple, self).__init__(name)

    def update(self):
        if True:
            return py_trees.common.Status.SUCCESS
        else:
            return py_trees.common.Status.FAILURE
# Other classes omitted…

has_apple = HasApple(name="Has apple")      #2
eat_apple = EatApple(name="Eat apple")      #2
sequence_1 = py_trees.composites.Sequence(name="Sequence 1", memory=True)
sequence_1.add_children([has_apple, eat_apple])                              #3

has_pear = HasPear(name="Has pear")         #4
eat_pear = EatPear(name="Eat pear")         #4
sequence_2 = py_trees.composites.Sequence(name="Sequence 2", memory=True)
sequence_2.add_children([has_pear, eat_pear])               #3

root = py_trees.composites.Selector(name="Selector", memory=True)
root.add_children([sequence_1, sequence_2])          #3

behavior_tree = py_trees.trees.BehaviourTree(root)     #5

py_trees.logging.level = py_trees.logging.Level.DEBUG
for i in range(1, 4):                                                         #6
    print("\n------------------ Tick {0} ------------------".format(i))
    behavior_tree.tick()                                                     #6

### Start of output
------------------ Tick 1 ------------------
[DEBUG] Selector             : Selector.tick()
[DEBUG] Selector             : Selector.tick() [!RUNNING->reset current_child]
[DEBUG] Sequence 1           : Sequence.tick()
[DEBUG] Has apple            : HasApple.tick()
[DEBUG] Has apple            : HasApple.stop(Status.INVALID->Status.SUCCESS)
[DEBUG] Eat apple            : EatApple.tick()
Eating apple
[DEBUG] Eat apple            : EatApple.stop(Status.INVALID->Status.SUCCESS)
[DEBUG] Sequence 1           : Sequence.stop()[Status.INVALID->Status.SUCCESS]
```

#1 Creates a class to implement an action or condition
#2 Creates the action and condition nodes
#3 Adds the nodes to their respective parents
#4 Creates the action and condition nodes
#5 Creates the whole behavior tree
#6 Executes one step/tick on the behavior tree

The code in listing 6.1 represents the behavior tree in figure 6.1. You can run this code as is or alter what the conditions return and then run the tree again. You can also change the behavior tree by removing one of the sequence nodes from the root selector.